

"""

reporte_excel.py
=====

Capa de EXPORTACION. Recibe DataFrames ya calculados y los escribe en un Excel multi-hoja con formato de bitacora auditable.

Separacion de responsabilidades: este modulo NO calcula ninguna metrica ni toma ninguna decision de seleccion. Si aqui hubiera un `if` que decidiera que variable se conserva, la trazabilidad se rompe (la decision no quedaria en la capa de calculo). Su unico trabajo es presentar.

Hojas generadas

00_Resumen	resumen ejecutivo y embudo de seleccion
01_Diagnostico_Inicial	perfil por columna + cabecera del dataset
02_Univariado	todas las columnas originales con sus flags
03_Bivariado	IV, Gini, score compuesto, exclusiones
04_Multivariado	asociacion, VIF, redundancias y seleccion final
05_Boruta	solo si usar_boruta = True
06_Seleccion_Final	lista final de variables para modelado
07_Parametros	configuracion y dependencias (reproducibilidad)
08_Bitacora_Log	log completo de la corrida
09_Diccionario	significado de cada columna de cada hoja
Anexos	matriz de asociacion, pares redundantes, tablas WOE

"""

```
from __future__ import annotations
```

```
from pathlib import Path
from typing import Any
```

```
import numpy as np
import pandas as pd
```

```
from .logging_utils import obtener_logger
```

```
LOGGER = obtener_logger("reporte")
```

```
# --- Paleta de la bitacora -----
```

```
COLOR_CABECERA = "1F3864"      # azul oscuro
COLOR_TITULO = "2E5C8A"
COLOR_OK = "C6EFCE"            # verde claro -> retenida / confirmada
COLOR_ELIMINADO = "FFC7CE"      # rojo claro -> eliminada / rechazada
COLOR_AVISO = "FFEB9C"         # ambar -> tentativa / advertencia
COLOR_NEUTRO = "F2F2F2"
```

```
#: Columnas cuyo valor 1 pinta la fila/celda de rojo (exclusiones).
```

```
FLAGS_NEGATIVOS = {
    "flg_eliminado_ceros", "flg_eliminado_variacion", "flg_exclusion",
    "flg_exclusion_multivariada", "flg_rechazada", "flg_no_evaluable",
    "flg_candidata_exclusion_temprana", "flg_varianza_cero", "flg_vif_alto",
    "flg_constante", "flg_std_baja", "flg_cv_bajo", "flg_categoria_dominante",
    "flg_percentiles_comprimidos", "flg_iv_bajo", "flg_gini_bajo",
}
```

```
#: Columnas cuyo valor 1 pinta de verde (selecciones).
```

```
FLAGS_POSITIVOS = {
    "flg_seleccionada_univariada", "flg_seleccionada_bivariada",
    "flg_seleccion_final", "flg_confirmada", "flg_seleccion_fases_1_3",
    "flg_supera_ruido",
}
```

```
#: Columnas cuyo valor 1 pinta de ambar (avisos que no eliminan).
```

```
FLAGS_AVISO = {"flg_sospecha_fuga", "flg_inestable_temporal", "flg_tentativa"}
```

```
#: Columnas que se formatean como porcentaje.
```

```
FORMATO_PORCENTAJE = {
    "pct_nulos", "pct_ceros", "pct_ceros_mas_nulos", "pct_ceros+nulos",
    "pct_unicos", "pct_valor_mas_frecuente", "pct_n", "tasa_evento",
    "target_medio", "tasa_aciertos",
}
```

```
#: Columnas que se formatean con 4 decimales.
```

```

FORMATO_DECIMAL = {
    "iv", "gini", "gini_bruto", "gini_woe", "gini_signo", "score_compuesto",
    "gini_normalizado", "iv_normalizado", "correlacion_maxima", "vif", "woe",
    "iv_bin", "dist_evento", "dist_no_evento", "psi_max", "iv_medio_periodo",
    "iv_min_periodo", "iv_max_periodo", "iv_std_periodo", "iv_cv_periodo",
    "asociacion", "score_a", "score_b", "iv_a", "iv_b", "gini_a", "gini_b",
    "piso_ruido_iv", "piso_ruido_gini", "umbral_iv_efectivo", "umbral_gini_efectivo",
    "icc_panel", "coef_variacion", "importancia_media",
    "importancia_sombra_max_media", "p_valor_confirmacion", "p_valor_rechazo",
}

# -----
# Utilidades de formato
# -----
def _sanear(df: pd.DataFrame) -> pd.DataFrame:
    """Deja el DataFrame en un estado que openpyxl pueda escribir sin fallar.

    Excel no admite infinitos, objetos arbitrarios ni celdas de mas de 32767
    caracteres; ademas los enteros de numpy fuera de rango revientan el
    escritor. Se normaliza todo aqui, una sola vez, en lugar de por hoja.
    """
    out = df.copy()
    out = out.replace([np.inf, -np.inf], np.nan)
    for col in out.columns:
        if out[col].dtype == object:
            out[col] = out[col].apply(
                lambda v: (str(v)[:32000] if not isinstance(v, (int, float, np.number, type(None)))
                           and not pd.isna(v) else v)
            )
    out.columns = [str(c)[:255] for c in out.columns]
    return out

def _escribir_hoja(writer: pd.ExcelWriter, nombre: str, df: pd.DataFrame,
                  titulo: str = "", startrow: int = 0) -> None:
    """Escribe un DataFrame en una hoja, tolerando errores por hoja."""
    if df is None or (isinstance(df, pd.DataFrame) and df.empty):
        df = pd.DataFrame({"aviso": [f"Sin datos para '{nombre}'."]})
    try:
        _sanear(df).to_excel(writer, sheet_name=nombre[:31], index=False, startrow=startrow)
    except Exception as exc: # noqa: BLE001 - una hoja rota no debe perder el resto
        LOGGER.error("No se pudo escribir la hoja '%s': %s", nombre, exc)
        pd.DataFrame({"error": [str(exc)]}).to_excel(
            writer, sheet_name=nombre[:31], index=False
        )

def _formatear_hoja(ws, df: pd.DataFrame, fila_encabezado: int = 1) -> None:
    """Aplica formato profesional a una hoja ya escrita."""
    from openpyxl.formatting.rule import CellIsRule
    from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
    from openpyxl.utils import get_column_letter

    if df is None or df.empty:
        return

    n_filas, n_cols = df.shape
    fuente_cab = Font(bold=True, color="FFFFFF", size=10)
    relleno_cab = PatternFill("solid", fgColor=COLOR_CABECERA)
    borde = Border(*(Side(style="thin", color="BFBFBF"),) * 4)

    # --- Encabezado -----
    for j in range(1, n_cols + 1):
        celda = ws.cell(row=fila_encabezado, column=j)
        celda.font = fuente_cab
        celda.fill = relleno_cab
        celda.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        celda.border = borde

    ws.row_dimensions[fila_encabezado].height = 30

```

```

ws.freeze_panes = ws.cell(row=fila_encabezado + 1, column=1)
if n_filas > 0:
    ws.auto_filter.ref = (
        f"A{fila_encabezado}:{get_column_letter(n_cols)}{fila_encabezado + n_filas}"
    )

# --- Ancho de columnas y formato numerico -----
for j, col in enumerate(df.columns, start=1):
    letra = get_column_letter(j)
    muestra = df[col].astype(str).head(200)
    ancho = max(len(str(col)) + 3, int(muestra.str.len().max() or 10) + 2)
    ws.column_dimensions[letra].width = min(max(ancho, 10), 55)

    nombre = str(col)
    if nombre in FORMATO_PORCENTAJE:
        fmt = "0.00%"
    elif nombre in FORMATO_DECIMAL:
        fmt = "0.0000"
    elif pd.api.types.is_float_dtype(df[col]):
        fmt = "#,##0.0000"
    elif pd.api.types.is_integer_dtype(df[col]):
        fmt = "#,##0"
    else:
        fmt = None

    if fmt:
        for i in range(fila_encabezado + 1, fila_encabezado + n_filas + 1):
            ws.cell(row=i, column=j).number_format = fmt

# --- Semaforo de flags -----
if nombre in FLAGS_NEGATIVOS | FLAGS_POSITIVOS | FLAGS_AVISO:
    color = (COLOR_ELIMINADO if nombre in FLAGS_NEGATIVOS
             else COLOR_OK if nombre in FLAGS_POSITIVOS else COLOR_AVISO)
    rango = f"{letra}{fila_encabezado + 1}:{letra}{fila_encabezado + n_filas}"
    ws.conditional_formatting.add(
        rango,
        CellIsRule(operator="equal", formula=["1"],
                    fill=PatternFill("solid", fgColor=color)),
    )

def _neutralizar_formulas(ws) -> int:
    """Fuerza a que todo texto se escriba como CADENA y nunca como formula.

    openpyxl marca como formula cualquier cadena que empiece por ``=``. Al abrir
    el archivo, Excel intenta compilarla; si no es una formula valida -y no lo
    es, por ejemplo, una linea de separadores ``====...`` del log, o un texto de
    negocio como ``=N/D``- Excel considera el libro danado y lo *repara*
    eliminando la celda, con el aviso:

        "Registros quitados: Formula de /xl/worksheets/sheetN.xml"

    El resultado es una bitacora con informacion perdida en silencio, que es
    justo lo contrario de lo que este proyecto persigue.

    Este libro no contiene ninguna formula legitima: todo son valores ya
    calculados. Por eso se puede convertir sin riesgo cualquier celda de tipo
    formula a texto plano, conservando el contenido exacto.

    La correccion se aplica en la capa de exportacion y no en el log, porque el
    problema no es de los separadores: lo dispararia igualmente cualquier valor
    del dataset del usuario que empiece por ``=``.

    Returns
    -----
    int
        Numero de celdas reconvertidas (se registra en el log).
    """
    convertidas = 0
    for fila in ws.iter_rows():
        for celda in fila:

```

```

        if celda.data_type == "f":
            celda.data_type = "s"
            convertidas += 1
    return convertidas

def _hoja_titulo(ws, texto: str, ancho: int) -> None:
    """Escribe una banda de titulo en la fila 1 de una hoja."""
    from openpyxl.styles import Alignment, Font, PatternFill

    ws.insert_rows(1)
    celda = ws.cell(row=1, column=1, value=texto)
    celda.font = Font(bold=True, size=12, color="FFFFFF")
    celda.fill = PatternFill("solid", fgColor=COLOR_TITULO)
    celda.alignment = Alignment(horizontal="left", vertical="center")
    ws.merge_cells(start_row=1, start_column=1, end_row=1, end_column=max(ancho, 2))
    ws.row_dimensions[1].height = 22

# -----
# Diccionario de columnas
# -----
def construir_diccionario() -> pd.DataFrame:
    """Glosario de cada columna relevante de la bitacora.

    Que el Excel se explique a si mismo es parte del requisito de auditabilidad:
    quien lo reciba dentro de seis meses no deberia necesitar el codigo fuente
    para entender que significa cada campo.
    """
    d = [
        # --- Diagnostico -----
        ("01_Diagnostico_Inicial", "rol", "Papel de la columna: TARGET, ID_ENTIDAD, TIEMPO, EXCLUIDA_MANUAL o CANDIDATO."),
        ("01_Diagnostico_Inicial", "tipo_inferido", "Familia detectada: NUMERICA, CATEGORICA, BOOLEANA o FECHA."),
        ("01_Diagnostico_Inicial", "pct_nulos", "Proporcion de valores ausentes sobre el total de filas."),
        ("01_Diagnostico_Inicial", "pct_ceros", "Proporcion de ceros (o cadena vacia en texto)."),
        ("01_Diagnostico_Inicial", "pct_ceros_mas_nulos", "Suma de ceros y nulos: masa de la variable sin contenido."),
        ("01_Diagnostico_Inicial", "n_unicos", "Cantidad de valores distintos (excluyendo nulos)."),
        ("01_Diagnostico_Inicial", "pct_valor_mas_frecuente", "Peso del valor modal. Cercano a 1 = variable casi constante."),
        ("01_Diagnostico_Inicial", "p25 / p50 / p75 / p90 / p99", "Percentiles de la distribucion. Si p25=p99 la variable es constante."),
        ("01_Diagnostico_Inicial", "iqr", "Rango intercuartilico p75-p25: dispersion robusta a valores extremos."),
        ("01_Diagnostico_Inicial", "coef_variacion", "std/|media|: dispersion RELATIVA, comparable entre variables."),
        ("01_Diagnostico_Inicial", "var_within", "Varianza promedio DENTRO de cada entidad: cuanto se mueve la variable dentro de cada entidad."),
        ("01_Diagnostico_Inicial", "var_between", "Varianza de las medias ENTRE entidades: diferencias estructurales entre entidades."),
        ("01_Diagnostico_Inicial", "icc_panel", "var_between/(var_between+var_within). ~1 = variable fija por entidad."),
        ("01_Diagnostico_Inicial", "flg_candidata_exclusion_temprana", "Aviso del diagnostico. NO elimina: la decision se toma a pesar de la exclusion."),
        # --- Univariado -----
        ("02_Univariado", "pct_ceros+nulos", "Criterio 1.1. Se elimina si supera el umbral configurado (95% por defecto)."),
        ("02_Univariado", "flg_eliminado_ceros", "1 = eliminada por exceso de ceros+nulos."),
        ("02_Univariado", "flg_eliminado_variacion", "1 = eliminada por baja variacion (cualquiera de los 5 subcriterios)."),
        ("02_Univariado", "flg_constante", "Subcriterio: un unico valor distinto."),
        ("02_Univariado", "flg_std_baja", "Subcriterio: desviacion estandar por debajo de umbral_std_minimo."),
        ("02_Univariado", "flg_cv_bajo", "Subcriterio: coeficiente de variacion por debajo de umbral_cv_minimo."),
        ("02_Univariado", "flg_categoria_dominante", "Subcriterio: una categoria concentra mas de umbral_dominancia del total de la variable."),
        ("02_Univariado", "flg_percentiles_comprimidos", "Subcriterio: IQR nulo y p25=p99 (la distribucion colapsa)."),
        ("02_Univariado", "flg_seleccionada_univariada", "1 = la variable pasa a la fase bivariada."),
        ("02_Univariado", "motivo_ceros / motivo_variacion", "Texto con la razon exacta de la eliminacion (trazabilidad)."),
        # --- Bivariado -----
        ("03_Bivariado", "iv", "Information Value: informacion acumulada bin a bin frente al target."),
        ("03_Bivariado", "clasificacion_iv", "Escala de Siddiqi: SIN_PODER <0.02, DEBIL <0.10, MEDIO <0.30, FUERTE >0.30."),
        ("03_Bivariado", "gini", "2*AUC-1. Capacidad de ORDENAR la muestra por riesgo. Se reporta en valor absoluto."),
        ("03_Bivariado", "gini_bruto", "Gini sobre los valores crudos (solo numericas): capta relaciones monotonas."),
        ("03_Bivariado", "gini_woe", "Gini sobre la proyeccion WOE: capta tambien relaciones no monotonas."),
        ("03_Bivariado", "gini_signo", "Gini con signo. Negativo = relacion inversa con el evento."),
        ("03_Bivariado", "metodo_binning", "Discretizacion aplicada. Documenta el tratamiento por tipo de variable."),
        ("03_Bivariado", "score_compuesto", "peso_gini*gini_norm + peso_iv*iv_norm. Es RELATIVO al conjunto evaluado."),
        ("03_Bivariado", "psi_max", "Population Stability Index maximo entre periodos. >0.25 = cambio poblacional significativo."),
        ("03_Bivariado", "iv_cv_periodo", "Coeficiente de variacion del IV entre periodos. Alto = poder predictivo variable a variable."),
        ("03_Bivariado", "piso_ruido_iv", "IV que alcanzaria una variable ALEATORIA con los mismos bins y la misma distribucion."),
        ("03_Bivariado", "piso_ruido_gini", "Gini que alcanzaria una variable aleatoria por azar, al nivel alpha_ruido."),
        ("03_Bivariado", "umbral_iv_efectivo", "max(umbral_iv_minimo, piso_ruido_iv): el baremo realmente aplicado."),
        ("03_Bivariado", "umbral_gini_efectivo", "max(umbral_gini_minimo, piso_ruido_gini): baremo realmente aplicado.")
    ]

```

```

("03_Bivariado", "flg_supera_ruido", "1 = la variable bate el azar en al menos una de las dos metricas."),
("03_Bivariado", "flg_sospecha_fuga", "IV anormalmente alto: revisar si la variable contiene informacion de"),
("03_Bivariado", "flg_exclusion", "1 = excluida en la fase bivariada. El motivo esta en motivo_exclusion_bi"),
# --- Multivariado -----
("04_Multivariado", "correlacion_maxima", "Asociacion mas alta de la variable con cualquier otra supervivie"),
("04_Multivariado", "variable_correlacionada", "Con que variable alcanza esa asociacion maxima."),
("04_Multivariado", "tipo_asociacion", "Medida usada: |correlacion| num-num, V de Cramer cat-cat, o eta num"),
("04_Multivariado", "vif", "Factor de inflacion de la varianza. >10 = colinealidad multiple severa."),
("04_Multivariado", "variable_que_la_desplaza", "Variable de mayor score compuesto que provoco la exclusion"),
("04_Multivariado", "flg_exclusion_multivariada", "1 = excluida por redundancia."),
("04_Multivariado", "flg_seleccion_final", "1 = supera las tres fases obligatorias."),
# --- Boruta -----
("05_Boruta", "boruta_status", "Confirmed / Tentative / Rejected respecto de las shadow features."),
("05_Boruta", "borutashap_status", "Idem con importancia SHAP; NO_APLICA si el motor usado no fue BorutaSha"),
("05_Boruta", "aciertos", "Numero de iteraciones en las que la variable supero a la mejor sombra."),
("05_Boruta", "alpha_bonferroni", "Nivel de significancia corregido por el numero de contrastes simultaneos"),
("05_Boruta", "flg_confirmada", "1 = importancia significativamente superior al ruido."),
("05_Boruta", "concordancia_con_fases_previas", "Contraste entre el veredicto de Boruta y el de las fases 1"),
# --- Anexos -----
("Anexo_WOE", "woe", "ln(dist_no_evento / dist_evento) del bin. 0 = el bin no aporta informacion."),
("Anexo_WOE", "iv_bin", "Contribucion del bin al IV total de la variable."),
("Anexo_WOE", "tasa_evento", "Proporcion de eventos dentro del bin."),
("Anexo_Pares_Redundantes", "criterio", "Regla exacta que decidio cual variable del par se conservo."),
]
return pd.DataFrame(d, columns=["hoja", "columna", "significado"])

# -----
# Exportacion principal
# -----
def exportar(resultados: dict[str, Any], ruta_salida: str | Path) -> Path:
    """Escribe el Excel completo de bitacora.

    Parameters
    -----
    resultados
        Diccionario producido por :func:`featsel.pipeline.ejecutar`.
    ruta_salida
        Ruta del archivo ``.xlsx``. Los directorios se crean si faltan.
    """
    ruta = Path(ruta_salida)
    ruta.parent.mkdir(parents=True, exist_ok=True)

    usar_boruta = bool(resultados.get("cfg_dict", {}).get("usar_boruta", False))
    boruta_meta = resultados.get("boruta_meta", {})

    # Orden de hojas: de lo ejecutivo a lo tecnico, y anexos al final.
    hojas: list[tuple[str, pd.DataFrame, str]] = [
        ("00_Resumen", resultados.get("resumen"),
         "RESUMEN EJECUTIVO DEL PROCESO DE SELECCION DE VARIABLES"),
        ("00_Embudo", resultados.get("embudo"),
         "EMBUDO DE SELECCION: COLUMNAS QUE ENTRAN Y SALEN EN CADA FASE"),
        ("01_Diagnostico_Inicial", resultados.get("diagnostico"),
         "FASE 0 - PERFIL ESTADISTICO DE CADA COLUMNA DEL DATASET"),
        ("01b_Diagnostico_General", resultados.get("general"),
         "FASE 0 - METRICAS DE CABECERA DEL DATASET Y DEL PANEL"),
        ("01c_Validacion_Panel", resultados.get("validacion"),
         "VALIDACIONES ESTRUCTURALES DEL PANEL (LLAVE ID+TIEMPO, BALANCE, TARGET)"),
        ("01d_Target_por_Periodo", resultados.get("target_por_periodo"),
         "DISTRIBUCION DEL TARGET A LO LARGO DE LA DIMENSION TEMPORAL"),
        ("02_Univariado", resultados.get("univariado"),
         "FASE 1 - PRUEBAS UNIVARIADAS (CEROS+NULOS Y BAJA VARIACION)"),
        ("03_Bivariado", resultados.get("bivariado"),
         "FASE 2 - PRUEBAS BIVARIADAS (INFORMATION VALUE, GINI Y SCORE COMPUESTO)"),
        ("04_Multivariado", resultados.get("multivariado"),
         "FASE 3 - PRUEBAS MULTIVARIADAS (REDUNDANCIA Y COLINEALIDAD)"),
    ]

    if usar_boruta:
        hojas.append((
            "05_Boruta", resultados.get("boruta"),

```

```

        f"FASE 4 - BORUTA / BORUTASHAP (MOTOR: {boruta_meta.get('motor', 'n/d')})",
    ))

hojas += [
    ("06_Seleccion_Final", resultados.get("seleccion_final"),
     "VARIABLES SELECCIONADAS PARA MODELADO"),
    ("06b_Descartadas", resultados.get("descartadas"),
     "VARIABLES DESCARTADAS CON LA FASE Y EL MOTIVO DE SALIDA"),
    ("07_Parametros", resultados.get("parametros"),
     "PARAMETROS DE LA CORRIDA (REPRODUCIBILIDAD)",
     "DEPENDENCIAS VERIFICADAS E INSTALADAS AUTOMATICAMENTE"),
    ("08_Bitacora_Log", resultados.get("log"),
     "REGISTRO CRONOLOGICO COMPLETO DE LA EJECUCION"),
    ("09_Diccionario", construir_diccionario(),
     "DICCIONARIO DE COLUMNAS DE LA BITACORA"),
    ("A1_Matriz_Asociacion", resultados.get("matriz_asociacion"),
     "ANEXO - MATRIZ DE ASOCIACION ENTRE VARIABLES SUPERVIVIENTES"),
    ("A2_Pares_Redundantes", resultados.get("pares_redundantes"),
     "ANEXO - PARES POR ENCIMA DEL UMBRAL Y DECISION TOMADA EN CADA UNO"),
    ("A3_Tablas_WOE", resultados.get("tablas_woe"),
     "ANEXO - TABLAS WOE POR VARIABLE (EVIDENCIA DEL CALCULO DEL IV)"),
]

LOGGER.info("Exportando bitacora a '%s' (%d hojas)...", ruta, len(hojas))

with pd.ExcelWriter(ruta, engine="openpyxl") as writer:
    for nombre, df, titulo in hojas:
        _escribir_hoja(writer, nombre, df, titulo)

    libro = writer.book
    total_neutralizadas = 0
    for nombre, df, titulo in hojas:
        hoja = nombre[:31]
        if hoja not in libro.sheetnames:
            continue
        ws = libro[hoja]
        datos = df if isinstance(df, pd.DataFrame) and not df.empty else pd.DataFrame(
            {"aviso": [f"Sin datos para '{nombre}'."]}
        )
        try:
            # Antes de cualquier formato: garantizar que no queden celdas
            # interpretadas como formula, que harian que Excel "repere" el
            # archivo eliminando su contenido.
            n_neutralizadas = _neutralizar_formulas(ws)
            if n_neutralizadas:
                total_neutralizadas += n_neutralizadas
                LOGGER.debug(
                    "Hoja '%s': %d celdas de texto reconvertidas de formula a cadena.",
                    hoja, n_neutralizadas,
                )
            _hoja_titulo(ws, titulo, datos.shape[1])
            _formatear_hoja(ws, datos, fila_encabezado=2)
            ws.sheet_properties.tabColor = (
                COLOR_CABECERA if nombre.startswith(("00", "06")) else "8EA9DB"
            )
        except Exception as exc:
            # noqa: BLE001 - el formato nunca debe perder datos
            LOGGER.warning("No se pudo formatear la hoja '%s': %s", hoja, exc)

    if libro.sheetnames:
        libro.active = 0

    if total_neutralizadas:
        LOGGER.info(
            "%d celdas de texto que empezaban por '=' se escribieron como cadena "
            "(evita que Excel repere el archivo eliminando su contenido).",
            total_neutralizadas,
        )

LOGGER.info("Bitacora escrita correctamente: %s (%.2f MB).",
            ruta.resolve(), ruta.stat().st_size / 1024**2)

```

```
return ruta.resolve()
```